

Administration de DB : Flashcard complet

? 01-Q : Cite un langage de 3eme génération

R : Java

? 02-Q : Globalement, quels sont les 3 étapes pour exécuter du code SQL ou PL/SQL dans un programme Java ?

R :

- 1- Etablir une connexion avec la BD
- 2- Envoyer les instructions SQL ou PL/SQL
- 3- Traiter le résultat

? 03-Q : Les packages suivant permettent la combinaison de Java et SQL. Quel est l'utilisé de ces packages :

- 1- java.sql
- 2- java.sql.DriverManager
- 3- java.sql.Connection
- 4- java.sql.Statement
- 5- java.sql.ResultSet

R :

- 1- **java.sql** : Accès et traitement des données dans une base de données
- 2- **java.sql.DriverManager** : gérer des sources JDBC
- 3- **java.sql.Connection** : Spécifier la connexion à la BD
- 4- **java.sql.Statement** : Exécuter et retourner le résultat SQL
- 5- **java.sql.ResultSet** : le résultat d'une requête SQL

? 04-Q : PL/SQL est destiné à quel SGBD ?

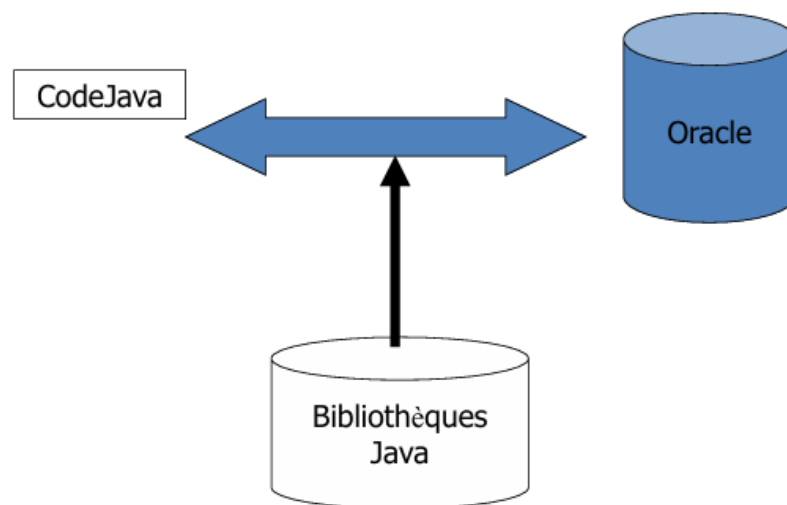
R : Oracle

? 05-Q : Les instructions suivant sont utilisé pour permettre la combinaison entre le langage Java et PL/SQL, quel est leur rôles ?

- 1- java.sql.Statement
- 2- java.sql.CallableStatement
- 3- stmt.execute ("create or replace function RAISESAL (name CHAR, raise NUMBER)return NUMBER is begin return raise + 100000; end;");

- 4- `CallableStatement cstmt = conn.prepareCall("{? = call RAISESAL (?, ?)}");`
- 5- `cstmt.registerOutParameter (1, Types.INTEGER);`
- 6- `cstmt.setString (2, nomPersonne);`
- 7- `cstmt.setInt (3, salaire);`
- 8- `ResultSet rs1 = cstmt.executeQuery();`
- 9- `nom1.setText(Integer.toString(cstmt.getInt(1)));`

R :



- 1- ***java.sql.Statement*** : Exécuter du SQL et récupérer le résultat de l'exécution
- 2- ***java.sql.CallableStatement*** : Exécution de procédures SQL stockées, donc du code PL/SQL
- 3- ***stmt.execute ("create or replace function RAISESAL (name CHAR, raise NUMBER)return NUMBER is begin return raise + 100000; end;");*** : Crée ou remplace une fonction avec ses paramètres et le type de retour
- 4- ***CallableStatement cstmt = conn.prepareCall("{? = call RAISESAL (?, ?)}");*** : Préparer l'appel à la fonction
- 5- ***cstmt.registerOutParameter (1, Types.INTEGER);*** : /identification de l'index de la valeur de retour ainsi que son type
- 6- ***cstmt.setString (2, nomPersonne);*** : identification du 1er paramètre
- 7- ***cstmt.setInt (3, salaire);*** : identification du 2eme paramètre
- 8- ***ResultSet rs1 = cstmt.executeQuery();*** : exécution de la procédure

- 9- `nom1.setText(Integer.toString(cstmt.getInt(1)))`; récupérer le résultat de la fonction

? 06-Q : Cite quelques SGBD sur le marché et l'entreprise propriétaire

R :

- 1- Oracle : Oracle
- 2- SQL Server : Microsoft
- 3- DB2 : IBM
- 4- Informix : IBM
- 5- Ingres : Computer Associates
- 6- Adaptive Server Enterprise : Sybase

? 07-Q : Quels sont les domaines d'application des SGBD ?

R :

- 1- Traitement Intelligents (Business Intelligence)
- 2- Complex Applications : tels que gestion financière
- 3- E-commerce
- 4- M-commerce : applications mobiles
- 5- Metadata Services
- 6- XML services

? 08-Q : Quels sont les technologies intégrées dans ces SGBD ?

R :

- 1- Availability
- 2- Ease of Use
- 3- Interoperability
- 4- Security

? 09-Q : Quels sont les limites des SGBD-R ?

R : Limites des SGBD-R

- 1- Le modèle de données est trop simple et ne permet pas de représenter facilement les entités du monde réel qui sont souvent plus complexes qu'une relation.
- 2- Le modèle relationnel ne permet pas de représenter explicitement les différents types de liens sémantiques qui peuvent lier des entités : composition, généralisation / spécialisation, association...

- 3- L'incompatibilité des LMD relationnels et des langages de programmation :
- Les LMD sont déclaratifs et fournissent en résultat un ensemble de tuples, alors que les langages de programmation sont impératifs et travaillent sur un élément à la fois
 - Les types de données manipulés par les langages de programmation sont plus complets et plus complexes que ceux des LMD relationnels.

? 10-Q : Comment sont nées les Bases de données orientées (BDO) ?

R : Les bases de données orientées objets (BDO) sont nées de la convergence de deux domaines :

- 1- Les bases de données
- 2- Les langages de programmation orientés objets, tels que Eiffel, Smalltalk, C ++, Java...

? 11-Q : Quels sont les 4 points qui caractérisent les BDO ?

R : Les 4 points :

- 1- Un modèle de données qui permet de représenter des structures de données complexes
- 2- Les données et les traitements ne sont plus séparés. La dynamique (les méthodes) fait partie de la déclaration des objets
- 3- L'héritage
- 4- Tout objet possède une identité qui le distingue de tout autre objet, même s'ils ont la même valeur

? 12-Q : Quels sont les deux types de liens qui existent entre les objets ?

R : Les deux types de liens :

- 1- Les liens de composition : "tel objet est composé de tel(s) objet(s)".
- 2- Les liens de généralisation / spécialisation

? 13-Q : Donne un exemple de lien de composition et généralisation/spécialisation ?

R :

Lien de composition

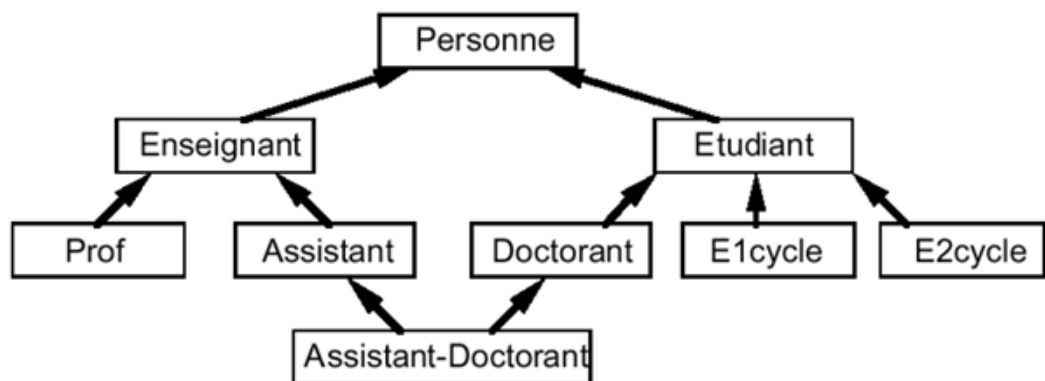
CLASS Etudiant2

```
{ num : INT ;  
  nom : STRING ;  
  prénoms : LIST STRING ;  
  cours-suivis : SET STRUCT {cours : Cours ;  
                              note : FLOAT } }
```

CLASS Cours

```
{nomC : STRING ; .... }
```

Graphe de généralisation/spécialisation des classes



? 14-Q : Explique le principe de l'Encapsulation ?

R :

- 1- L'accès aux données des objets se fait à travers des méthodes : on dit que les données sont "encapsulées"

- 2- Seules les méthodes de la classe et de ses sous-classes peuvent accéder directement à la valeur des objets.

? 15-Q : Quels sont les avantages de l'encapsulation ?

R : Avantages :

- 1- Simplifier le travail des programmeurs d'application en ne leur demandant que de connaître le minimum
- 2- Faciliter la maintenance des applications : on peut facilement changer la structure d'un objet...
- 3- Réutiliser au maximum le code des programmes : les méthodes sont écrites une fois pour toutes et utilisées par tous les programmes

? 16-Q : Quels sont les avantages de BDO ?

R : Avantages :

- 1- Ceux de l'approche orientée objets : réutilisation, maintenance facilitée...
- 2- Modèle de données riche
- 3- Prise en compte de la dynamique
- 4- Beaucoup moins d'incompatibilité entre le LMD et le langage de programmation.

? 17-Q : Quels sont les problèmes des BDO ?

R : Problèmes :

- 1- La simplicité du modèle relationnel est perdue.
- 2- Pas de standard
- 3- Pas encore de méthode complète de conception des données et des méthodes associées.
- 4- Des travaux sont en cours sur les vues, les versions, l'évolution du schéma, les aspects temporels, etc

? 18-Q : Quels sont les 3 niveaux d'abstraction de l'ANSI et de Merise ?

R : Les 3 niveaux :

- 1- Niveau externe
- 2- Niveau conceptuel
- 3- Niveau interne

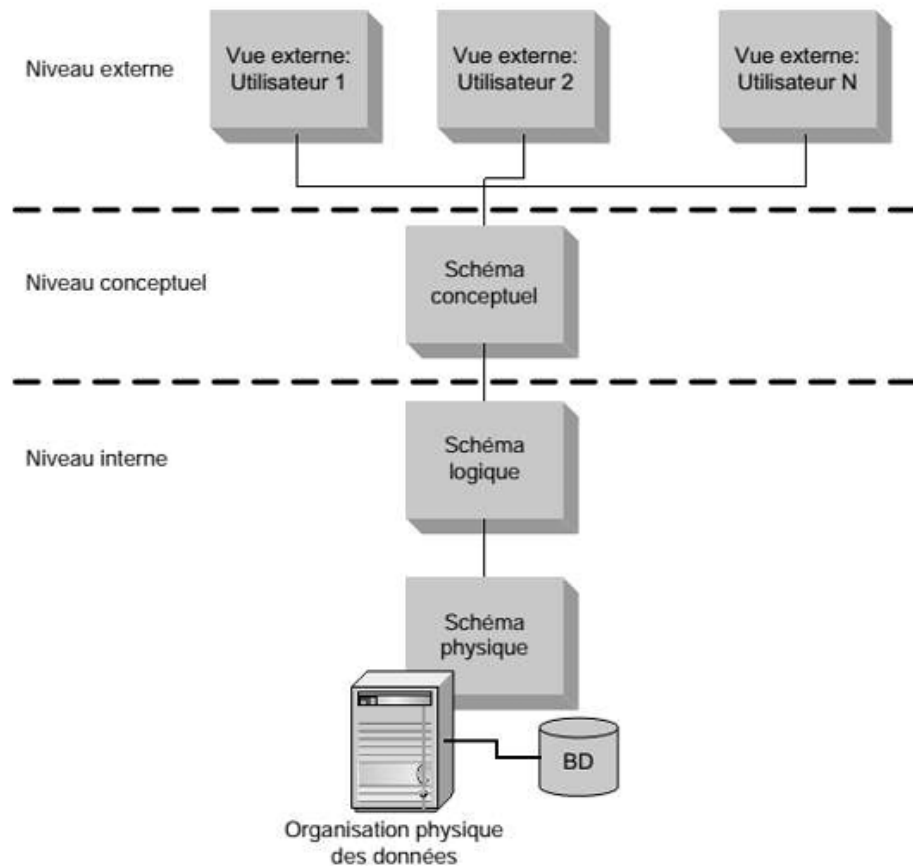


Fig1 : Les trois niveaux d'abstraction de l'ANSI et de merise aussi

? 19-Q : Fait une représentation de l'architecture client-serveur à deux niveaux

R : Architecture client-serveur à deux niveaux

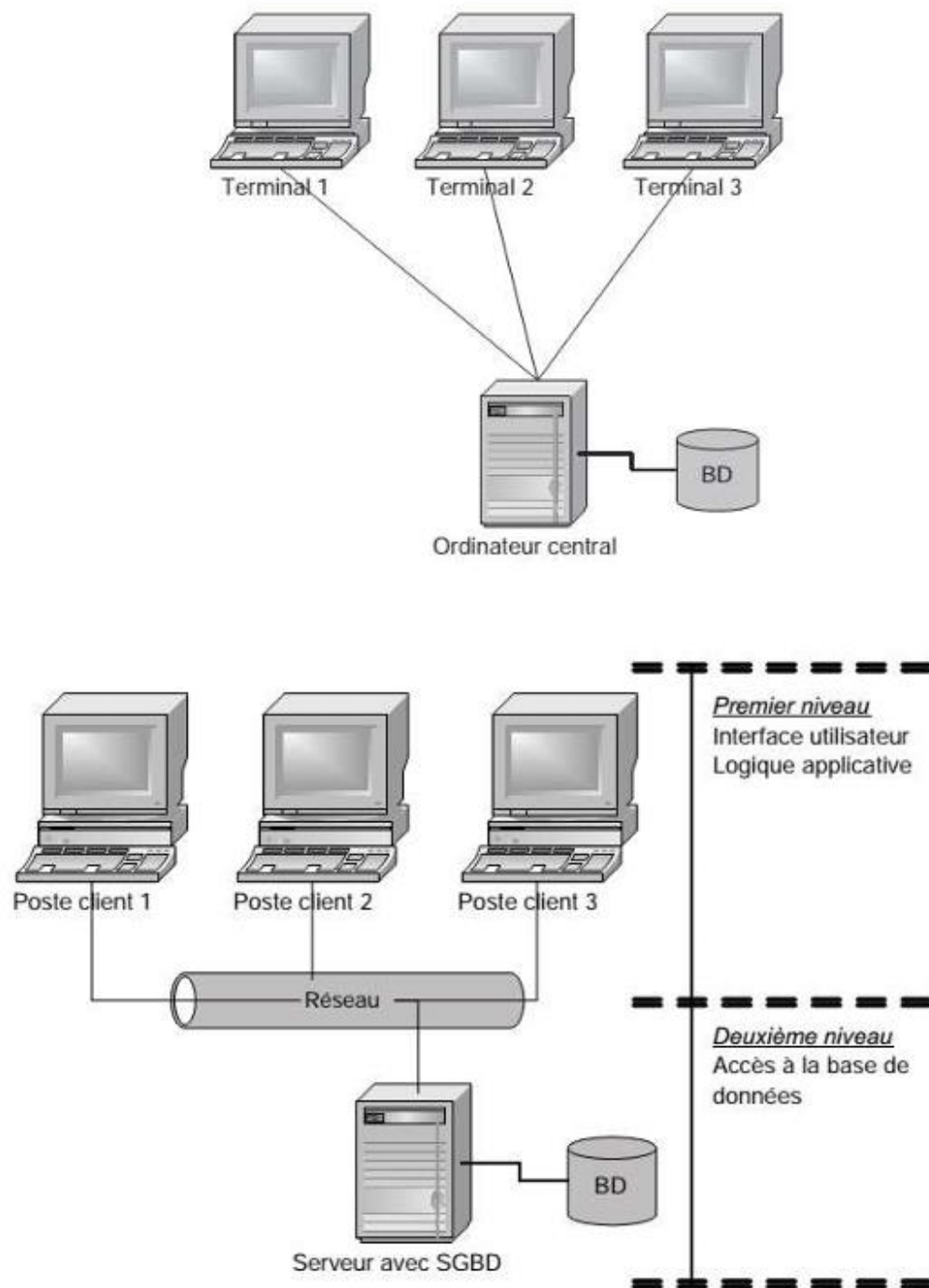


Fig2 : Architecture client-serveur à deux niveaux

? 20-Q : Fait une représentation de l'Architecture client-serveur à trois niveaux

R : Architecture client-serveur à trois niveaux

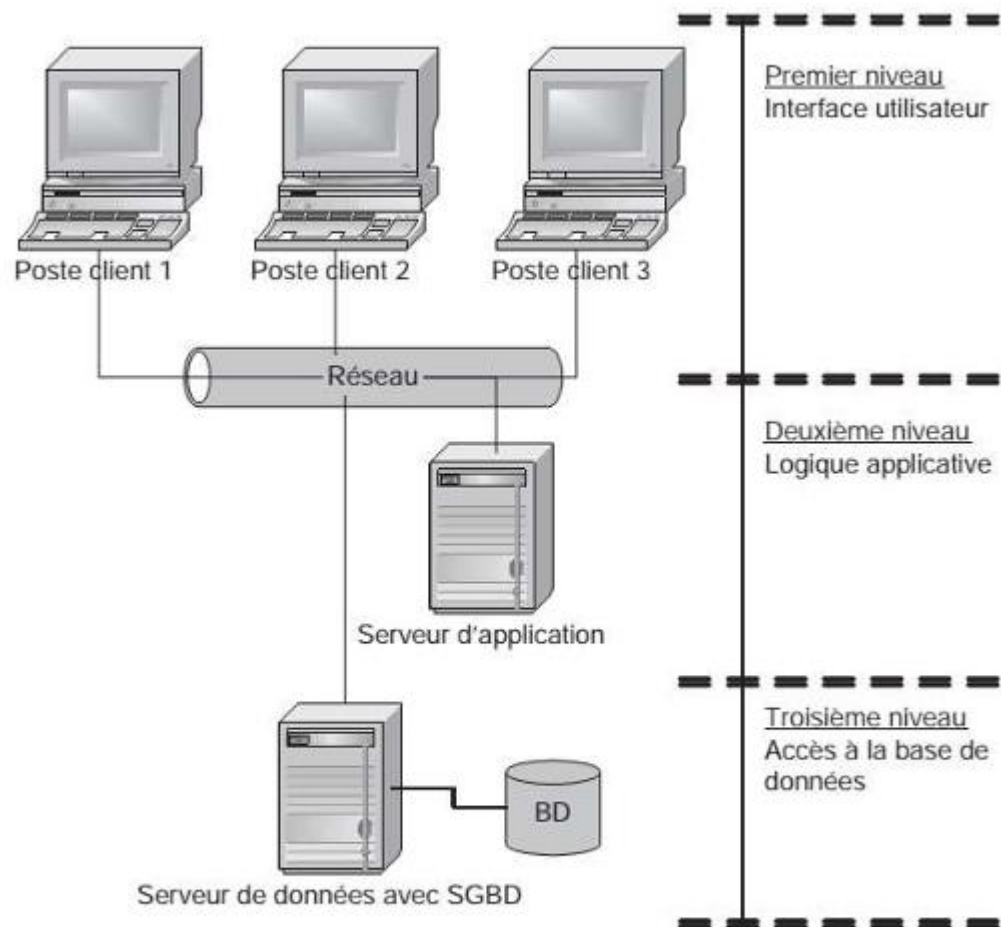


Fig3 : Architecture client-serveur à trois niveaux

? 21-Q : La gestion de la sécurité dans les bases de données, est une vaste problématique, qui peut être abordée sur différents volets. Quels sont ces volets ?

R : Parmi ces volets nous pouvons citer :

- 1-** Protection des données contre les accès non autorisés, à travers la mise en œuvre d'un mécanisme d'authentification, de définition et de vérification des droits d'accès.
- 2-** Protection des données contre les pannes, à travers la définition d'une politique de sauvegarde, et de restauration en cas de défaillance matérielle ou logicielle.

? 22-Q : L'affirmation suivante est-elle vraie ou fausse ?

Une base de données ne peut être manipulée que par un ensemble défini d'utilisateurs. Chaque utilisateur doit s'authentifier à travers un mot de passe.

R : Vrai

? 23-Q : Lors de la création d'un utilisateur, des espaces de tables de données, et de tables temporaires doivent lui être attribués, pourquoi ?

R : C'est pour lui permettre éventuellement de créer des données. A chaque espace de nom on lui fixe un quota d'octet qu'il ne peut pas dépasser.

? 24-Q : Donne la syntaxe SQL qui permet de créer un utilisateur Samba qui peut utiliser 12 Mo sur l'espace de tables nommé « outils » et 4 Mo sur l'espaces de tables temporaires nommé « temp ». ?

R :

```
CREATE USER Samba IDENTIFIED BY Sam2015
DEFAULT TABLESPACE outils
TEMPORARY TABLESPACE temp
QUOTA 12M ON outils
QUOTA 4M ON temp;
```

? 25-Q : Quel commande permet de modifier le mot de passe d'un utilisateur, de le supprimer uniquement, de la supprimer ainsi que tous les objets qu'il a créés ?

R :

```
ALTER USER <nom-utilisateur> IDENTIFIED BY
<nouveau-mot-de-passe>;
```

Ou de supprimer:

```
DROP USER <nom-utilisateur>;
```

Ou encore de le supprimer, ainsi que tous les objets qu'il a crée:

```
DROP USER <nom-utilisateur> CASCADE;
```

? 26-Q : Dans les SGBD, c'est quoi un catalogue ?

R : C'est des tables ou vue qui sont entretenue par le SGBD et qui lui permet de conserver des informations sur les utilisateurs. Ces catalogues peuvent être interrogés de la même façon que les tables utilisateurs, à travers la commande **SELECT**.

? 27-Q : Cite quelques catalogues et leur role ?

R :

CATALOGUE	INFORMATION
ALL_USERS	Liste des utilisateurs
USER_USERS	Utilisateur courant
USER_TABLES	Les tables appartenant à l'utilisateur courant
USER_TS_QUOTAS	Les quotas d'espaces tables de l'utilisateur courant
USER_OBJECTS	Les objets appartenant à l'utilisateur courant

? 28-Q : La simple création d'un utilisateur suffit-il à lui accordé les droits d'accès, de création ou de modification des schémas ou des données ?

R : NON, il faut lui accorder les privilèges nécessaires pour ces actions à travers la commande **GRANT**

? 29-Q : Quels sont les étapes basic pour accorder des privilèges à l'utilisateur Samba ?

R : Etapes :

- 1- Lui permettre de se connecter à la base de données, c'est-à-dire de créer une session sur le SGBD : **GRANT CREATE SESSION TO Samba;**
- 2- Lui permettre de créer des tables : **GRANT CREATE TABLE TO Samba;**
- 3- Notre utilisateur peut se connecter en utilisant la commande : **CONNECT Samba/sam2015;**

Il peut créer un schéma relationnel, et dispose de tous les privilèges sur ce schéma : modifications structurelles (ALTER TABLE, DROP TABLE), et manipulation des données (SELECT, INSERT, DELETE, UPDATE)

? 30-Q : Quel commande permet de donner tous les privilèges système à un utilisateur ?

R: **GRANT ALL PRIVILEGES TO <nom-utilisateur>;**

? 31-Q : Quel commande l'utilisateur Samba doit utiliser pour permettre à l'utilisateur Salif d'interroger la table Etudiant qu'il a créée ?

R: **GRANT SELECT ON Etudiant TO Salif;**

La syntaxe de base est donc : **GRANT <privilège> ON <table> TO <utilisateur>;**

? 32-Q : Une fois connecté dans sa propre session, comment l'utilisateur Salif peut consulter la table Etudiant de Samba ?

R: **SELECT * FROM Samba.Etudiant;**

? 33-Q : hormis le privilège SELECT, quels sont les autres privilèges que tu connais ?

R : **ALTER, INSERT, UPDATE, DELETE**

? 34-Q : Quel est la syntaxe pour ôter un privilège (système ou objet) à un utilisateur et donne des exemples ?

R : Syntaxe et exemple :

REVOKE <privilège> FROM <nom-utilisateur>;
(privilège système)

REVOKE <privilège> ON <nom_table> FROM <nom-utilisateur>; *(privilège objet)*

Exemples:

REVOKE CREATE TABLE FROM Samba;

REVOKE SELECT ON Etudiant FROM Salif;

? 35-Q : C'est quoi un rôle et quel est son intérêt ?

R : Un rôle peut être vu comme un groupe d'utilisateurs possédant un certain nombre de privilèges. Il permet d'éviter des tâches répétitives lors de la définition des privilèges d'utilisateurs.

? 36-Q : Quel est la syntaxe pour créer un rôle ?

R : *CREATE ROLE <nom-rôle>;*

? 37-Q : Donne des exemples de création de rôle et d'attribution de privilège à ces rôles ?

R : Exemple :

```
CREATE ROLE BD_Admin;  
CREATE ROLE Samba_BD_user;  
GRANT CREATE SESSION, CREATE TABLE TO BD_Admin;  
GRANT SELECT ON Etudiant TO Samba_BD_user;
```

? 38-Q : Comment mapper un utilisateur nommé Sirine sur le premier rôle et l'utilisateur Salif sur le deuxième rôle ?

R : Commande :

*1- GRANT BD_Admin TO Sirine;
2- GRANT Samba_BD_user TO Salif;*

? 39-Q : Cite quelques catalogues sur les privilèges ?

R : Catalogues :

Catalogue	Information
USER_SYS_PRIVS	Liste des privilèges systèmes de l'utilisateur courant
USER_TAB_PRIVS_RECD	Liste des privilèges objets de l'utilisateur courant
USER_ROLES_PRIVS	Liste des rôles accordés à l'utilisateur courant

? 40-Q : Quel sigle fait référence à la sécurité des données ou contrôle des données en SQL ?

R: DCL (Data Control Language)

? 41-Q : Le langage de requête de base de données SQL se base sur une forme de requête théorique pour exprimer les besoins. Comment se nomme cette forme de requête théorique ?

R : Algèbre relationnelle

? 42-Q : Quels sont les 3 langages qui compose SQL ?

R : Les 3 langages:

- 1- DDL (Data Definition Language) : Composant autorisant la définition (création) de composantes de base de données. Par exemple : les tables.
- 2- DML (Data Manipulation Language) : Composant permettant le traitement de l'information de la base de données. Par exemple : extraction, ajout et modification de données.
- 3- DCL (Data Control Language) : Composant fournissant la sécurité interne de la base de données. Par exemple : usagers, mot de passe, permissions.

? 43-Q : Pourquoi dit-on que SQL est un langage non-procédural ?

R : Parce que les expressions SQL stipulent ce qui doit être fait et non comment cela doit être fait.

? 44-Q : Quels sont les instructions pour la définition des données (DDL) ?

R : Instructions:

- 1- **CREATE TABLE** : Créer des tables
- 2- **ALTER TABLE** : Modifier une table (Ex : ajouter une nouvelle colonne, supprimer une colonne)
- 3- **DROP TABLE** : Supprimer une table
- 4- **CREATE INDEX** : Créer un index
- 5- **CREATE TRIGGER**

? 45-Q : Quel est la syntaxe pour créer une table ?

R : Syntaxe :

```
CREATE TABLE nom_table  
(nom_col1 TYPE1,  
nom_col2 TYPE2,  
...)
```

? 46-Q : Cite quelques contraintes qui peuvent être défini sur les champs lors de la création d'une table ?

R : Contraintes :

- 1- **PRIMARY KEY** : Désigner une clé primaire
- 2- **REFERENCES** : Désigner une clé étrangère, établissant une relation entre 2 tables
- 3- **UNIQUE** : Chaque ligne de la table doit avoir une valeur différente ou NULL pour cette (ou ces) colonne(s).
- 4- **NOT NULL** : La valeur de la colonne ne doit pas être NULL
- 5- **CHECK** : Restreindre les valeurs autorisées à une plage spécifiée.

? 47-Q : Il y a 2 types de clauses de contraintes dans une instruction CREATE TABLE, Cite et décris-les ?

R : Clauses de contraintes:

- 1- **Contrainte de colonne unique** :
 - a. Elle est utilisée à l'intérieur d'une définition de colonne.
 - b. Elle sert à définir une contrainte qui ne s'applique qu'à une seule colonne, celle incluse dans la définition.

```
PubNom TEXT(100) NOT NULL
```

ou bien

```
PubNom TEXT(100) Constraint valeurNulle NOT NULL
```

```
CREATE TABLE EDITEURS
```

```
(PubID TEXT(10) Constraint cléPrimaire Primary Key,  
PubNom TEXT(100) Not Null,  
PubPhone TEXT(20))
```

```
CREATE TABLE LIVRES
(ISBN TEXT(13) CONSTRAINT ClePrimaire PRIMARY
KEY,
Titre TEXT(100) NOT NULL,
Prix MONEY CONSTRAINT contPrix CHECK (Prix>19),
PubID TEXT(10),CONSTRAINT cleEtrangere FOREIGN
KEY (PubID) REFERENCES EDITEURS (PubID));
```

- 2- **Contrainte de colonne multiple** : Les contraintes multi-colonne servent lorsque plusieurs colonnes sont en jeu dans la contrainte. Clé primaire double, clé étrangère double, etc.

```
CONSTRAINT Nom
[CHECK(expressionDeContrainte)| PRIMARY KEY
(NomColonne,...) |
UNIQUE(NomColonne,...) |
FOREIGN KEY (ColonneReference,...)
REFERENCES
TableReference[(ColonneReference,...)]]
```

Exemple : Création du schéma de table Personne où chaque personne est identifiée par son nom et prénom (par exemple)

```
CREATE TABLE Personne
(Nom TEXT(10)
Prenom TEXT(100),
Adresse TEXT(20)
CONSTRAINT ClePrimaire PRIMARY KEY (Nom,
Prenom));
```

Les paramètres de l'instruction CREATE TABLE sont un nom de table suivi par une ou plusieurs définitions de colonnes, suivies par 0, une ou plusieurs contraintes multi-colonnes.

Définition de colonne

DefinitionColonne ::= NomColonne
TypeDonnee[(taille)]
[ContrainteColonneSimple]

3-

? 48-Q : Si on spécifie une clé étrangère, l'intégrité référentielle est garantie, cela veut dire quoi ?

R : Cela veut dire que le parent (la table qui contient la clé primaire) ne peut pas être supprimé tant qu'il existe au moins une table enfant (la table qui contient la clé étrangère) qui lui fait référence.

? 49-Q : Pour garantir l'intégrité référentielle, il faut spécifier ce qu'il faudrait faire en modifiant ou supprimant une table parente. Pour cela SQL nous offre certains outils, cite-les

R : Outils pour garantir l'intégrité référentiel :

1- ON UPDATE CASCADE

2- ON DELETE CASCADE

3- ON DELETE SET NULL

Exemple d'utilisation :

1- CONSTRAINT NomIndex FOREIGN KEY (PubID) REFERENCES EDITEURS (PubID) ON UPDATE CASCADE

2- CONSTRAINT NomIndex FOREIGN KEY (PubID) REFERENCES EDITEURS (PubID) ON DELETE CASCADE

3- CONSTRAINT NomIndex FOREIGN KEY (PubID) REFERENCES EDITEURS (PubID) ON DELETE SET NULL

? 50-Q : Quel est la syntaxe pour l'instruction ALTER TABLE ?

R : Syntaxes:

ALTER TABLE

NomTable

ADD COLUMN NomColonne Type[(grandeur)][contrainte]

DROP COLUMN NomColonne |

ADD CONSTRAINT ContrainteMultiColonne

DROP CONSTRAINT ContrainteMultiColonne

? 51-Q : Donne la requête qui permet d'ajouter la colonne prixCoutant de type CURRENCY à la table Livre ?

R : *ALTER TABLE Livre ADD COLUMN prixCoutant CURRENCY;*

? 52-Q : Quel est la syntaxe pour supprimer une table ou l'index d'une table ?

R : *DROP TABLE NomTable | DROP INDEX NomIndex ON NomTable;*

? 53-Q : Quels sont les langages de manipulation de données (DML) ?

R : DML:

1- *SELECT*

2- *UNION*

3- *UPDATE*

4- *DELETE*

5- *INSERT INTO*

6- *SELECT INTO*

? 54-Q : Quel est la syntaxe de l'instruction SELECT ?

R : Instruction SELECT (projection Π en algèbre relationnelle) :

```
SELECT [prédicat] Colonne,...  
FROM ExpressionTable  
[WHERE Condition]  
[GROUP BY CritèreGroupe]  
[HAVING Condition]  
[ORDER BY CritèreOrdre]
```

1- Prédicat : Il nous permet de faire des choix quant aux lignes identiques qui seraient retournées par la requête. On peut utiliser les prédicats suivants : **ALL**, **DISTINCT**.

- ALL (par défaut) retourne toutes les lignes lues sans exception.
- DISTINCT ne retourne que les lignes lues qui sont différentes. S'il y a des lignes pareilles, seule la première apparaîtra.

🍌 Par exemple:

```
SELECT EDITEURS.PubNom  
FROM EDITEURS, LIVRES WHERE  
    EDITEURS.PubID = LIVRES.PubID;
```

- ici, puisque qu'un éditeurs a plusieurs livres, son nom apparaîtra plusieurs fois.

```
SELECT DISTINCT PubNom  
FROM EDITEURS, LIVRES WHERE  
    EDITEURS.PubID = LIVRES.PubID;
```

- Ici, même si l'éditeurs a plusieurs livres, son nom n'apparaîtra qu'une fois.

2- Colonne : La colonne ou combinaisons de colonnes à retourner

```
SELECT titre, livre.pubID, editeur.pubID FROM livre, Editeur
```

```
SELECT titre, livre.pubID AS LID, editeur.pubID AS EID FROM  
livre, Editeur
```

- 3- ExpressionTable : Cette clause spécifie les tables (ou requêtes) dont l'instruction SELECT extrait ses lignes. ExpressionTable peut être un nom de table unique, plusieurs noms de tables séparés par des virgules, ou une clause de jointure. Elle peut aussi avoir un **AS** *NomAlias*.

```
SELECT * FROM Auteur;  
SELECT * FROM Auteur,Editeur;  
SELECT * FROM ClauseJointure;  
SELECT * FROM Auteur AS A, Editeur AS B;
```

- 4- WHERE (en algèbre relationnelle : Restriction σ) : permet de spécifier des conditions quant aux lignes retournées. Les expressions peuvent comporter des noms de colonne, constantes, comparaisons (=, , <=, >=, <>, BETWEEN), des relations (AND, OR, XOR, NOT) et des fonctions.

? 55-Q : Donne des exemples de requête SQL-algèbre relationnelle sur SELECT ?

R : Exemple :

- $\Pi_{solde, nom}$ (compte) devient
Select solde, nom from compte.
- $\sigma_{solde > 2200}$ (compte) devient
Select * from compte where solde > 2200
- $(\Pi_{nom} (\sigma_{solde > 2200} (compte)))$ devient
Select nom from compte where solde > 2200
- $\rho_{client}(numclient, nomclient, soldeclient)$ (compte (no-compte, nom, solde))
devient
Select no-compte AS numclient, nom AS nomclient, solde AS soldeclient
From compte AS client.

? 56-Q : Quel est l'utilité des Jointures en SQL ?

R : Les jointures permettent de traiter les données de 2 ou plusieurs tables dans la même requête.

? 57-Q : Quel élément dans l'algèbre relationnelle fait référence au Jointure en SQL ?

R : Produit cartésien (Ex : livre x Editeur)

? 58-Q : Parle-moi de Jointure interne

R : Jointure interne:

- Jointure interne(naturelle et Thêta)
 - La syntaxe d'une clause de jointure interne est la suivante:
Select colonne,....
From Table1, Table2
WHERE Table1.Colonne1 = Table2.Colonne1
[{AND|OR condition jointure thêta...}]
 - Ce type de jointure permet de trouver exclusivement les dossiers des 2 tables qui correspondent par les colonnes mentionnées.

? 59-Q : Donne des exemples de code SQL/Algèbre relationnelle sur la Jointure interne ?

R : Exemple :

```
● client |x| compte = ( $\sigma_{\text{Client.nom} = \text{Compte.nom}}$  (client x compte))
Select *
From client, compte
Where client.nom = compte.nom

●  $\sigma_{\text{Compte.solde} > 1800}$  (client |x| compte)).
Select *
From client, compte
Where (client.nom = compte.nom) and (compte.solde > 1800)

●  $\Pi_{\text{client.rue, compte.solde}}$  (client |x| $\theta$  compte) avec  $\theta = (\text{Compte.solde} > 1800)$ 
Select client.rue, compte.solde
From client, compte
Where (client.nom = compte.nom) and (compte.solde > 1800)
```

? 60-Q : Parle moi de la Jointure externe

R : Jointure externe:

- Jointure externe
 - Pour exprimer la jointure externe, il faut placer un (+) au niveau du champs où la jointure externe serait appliquée.
 - En SQL, on ne peut exprimer la jointure externe. On exprime la jointure externe gauche ou droite.
 - Le (+) se place sur le champ de droite pour une jointure externe gauche et à gauche pour une jointure externe droite.
 - ```
select A.col1, A.col2, B.col1, B.col2
from A, B
where A.col1 = B.col1(+);
```

● Editeur  $\rightarrow$  Livre

Select \*

From Editeur, Livre

Where Editeur.NoE = Livre.NoE (+)

●  $\Pi_{\text{nom, no-dep}}$  (employe  $\rightarrow$  departement)

Select employe.nom, departement.no-dep

From employe, departement

Where employe.nas = departement.nas-g (+)

● Editeur  $\rightarrow$  Livre

Select \*

From Editeur, Livre

Where Editeur.NoE (+) = Livre.NoE

? 61-Q : Donne un exemple de cas où on pourrai utilisé la clause GROUP BY

R : Utilisation de GROUP BY :

| Company   | Amount |
|-----------|--------|
| W3Schools | 5500   |
| IBM       | 4500   |
| W3Schools | 7100   |

« SELECT Compagnie, SUM(Montant) FROM Ventas » est refusée alors que:  
“SELECT Company,SUM(Amount) FROM Sales GROUP BY Company”  
donne...

| Company   | SUM(Amount) |
|-----------|-------------|
| W3Schools | 12600       |
| IBM       | 4500        |

? 62-Q : Parle-moi de la clause HAVING

R :

### ● Clause HAVING CritereGroupe

- La clause HAVING est le filtrage des groupes de l'algèbre.
- La clause HAVING a été introduite car le WHERE ne peut pas être utilisé avec les fonctions d'agrégation comme SUM.
- Sans HAVING, il ne serait pas possible de tester les conditions portant sur le résultat d'une fonction

- La clause HAVING n'affecte que les valeurs qui sont affichées
- La clause HAVING ne s'applique qu'après le regroupement.
- Le critère du WHERE s'applique sur un champ (et non sur une agrégation)
- Le critère HAVING s'applique sur un champ regroupé (inclus dans la clause GROUP BY) ou un champ agrégé (inclus dans une fonction d'agrégation).



🍌 Par exemple:

```
SELECT Editeurs.Pubnom, AVG(Prix) AS PrixMoyen
FROM Editeurs, Livres WHERE Editeurs.PubID =
 Livres.PubID
GROUP BY Editeurs.PubNom HAVING AVG(Prix) < 25.00;
```

| Company   | Amount |
|-----------|--------|
| W3Schools | 5500   |
| IBM       | 4500   |
| W3Schools | 7100   |

SELECT Company, SUM(Amount) FROM Sales GROUP BY Company HAVING SUM(Amount)>10000 donne...

| Company   | SUM(Amount) |
|-----------|-------------|
| W3Schools | 12600       |

? 63-Q : Comment utilisé la clause ORDER BY ?

R :

| Company   | OrderNumber |
|-----------|-------------|
| Seqa      | 3412        |
| ABC Shop  | 5678        |
| W3Schools | 2312        |
| W3Schools | 6798        |

SELECT Company, OrderNumber FROM Orders ORDER BY Company (ordre alphabétique)

| Company   | OrderNumber |
|-----------|-------------|
| ABC Shop  | 5678        |
| Seqa      | 3412        |
| W3Schools | 6798        |
| W3Schools | 2312        |

**SELECT Company, OrderNumber FROM Orders ORDER BY Company DESC, OrderNumber ASC** (trie les compagnies par ordre alphabétique décroissant et les OrderNumber par ordre numérique croissant)

| <b>Company</b> | <b>OrderNumber</b> |
|----------------|--------------------|
| W3Schools      | 2312               |
| W3Schools      | 6798               |
| Seqa           | 3412               |
| ABC Shop       | 5678               |

**? 64-Q : Parle-moi de la clause UNION**

**R : Clause UNION :**

### L'Opération UNION

- L'opération UNION est utilisée pour créer l'union de 2 ou plusieurs tables. La syntaxe est:  
Requete {UNION [ALL] Requete},...
- Où Requete est soit une instruction SELECT ou le nom d'une requête stockée (sera vue au prochain cours).
- Par défaut, les doublons sont triés entre les deux requêtes et n'apparaissent qu'une seule fois.
- L'option ALL force à inclure tous les enregistrements et donc de voir les doublons.
- ALL améliore les performances car il n'y a pas de tri de doublon.

## ● Par exemple

```
SELECT * FROM Livres
```

```
UNION ALL
```

```
SELECT * FROM NewLivres WHERE Prix > 25.00
```

```
ORDER BY Titre;
```

## ● Notes

- Toutes les requêtes dans une opération UNION doivent retourner le même nombre de champs.
- Les colonnes sont combinées dans l'union selon leur ordre dans la clause de requêtes et non par leur nom

# EXEMPLE

Résultats de l'union

Table Livres

| ISBN          | Titre           | PubID | Prix  |
|---------------|-----------------|-------|-------|
| 0-103-45678-9 | Iliad           | 1     | 25,00 |
| 0-11-345678-9 | Moby Dick       | 3     | 49,00 |
| 0-12-333433-3 | On Liberty      | 1     | 25,00 |
| 0-123-45678-0 | Ulysses         | 2     | 34,00 |
| 0-12-345678-9 | Jane Eyre       | 3     | 49,00 |
| 0-321-32132-1 | Balloon         | 3     | 34,00 |
| 0-55-123456-9 | Main Street     | 3     | 25,00 |
| 0-555-55555-9 | MacBeth         | 2     | 12,00 |
| 0-91-045678-5 | Hamlet          | 3     | 20,00 |
| 0-91-335678-7 | Fairie Queene   | 1     | 15,00 |
| 0-99-777777-7 | King Lear       | 2     | 49,00 |
| 0-99-999999-9 | Emma            | 1     | 20,00 |
| 1-1111-1111-1 | C++             | 1     | 30,00 |
| 1-22-233700-0 | Visual Basic    | 1     | 25,00 |
| 1-56592-297-2 | Access Database | 3     | 30,00 |

| ISBN          | Titre     | PubID | Prix  |
|---------------|-----------|-------|-------|
| 0-103-45258-9 | Corba     | 1     | 25,00 |
| 0-11-354678-9 | Paris     | 3     | 29,00 |
| 0-12-333883-3 | SQL       | 1     | 15,00 |
| 0-12-335671-9 | John Bull | 3     | 39,00 |

| ISBN          | Titre           | PubID | Prix |
|---------------|-----------------|-------|------|
| 1-56592-297-2 | Access Database | 3     | 30   |
| 0-321-32132-1 | Balloon         | 3     | 34   |
| 1-1111-1111-1 | C++             | 1     | 30   |
| 0-99-999999-9 | Emma            | 1     | 20   |
| 0-91-335678-7 | Fairie Queene   | 1     | 15   |
| 0-91-045678-5 | Hamlet          | 3     | 20   |
| 0-103-45678-9 | Iliad           | 1     | 25   |
| 0-12-345678-9 | Jane Eyre       | 3     | 49   |
| 0-12-335671-9 | John Bull       | 3     | 39   |
| 0-99-777777-7 | King Lear       | 2     | 49   |
| 0-555-55555-9 | MacBeth         | 2     | 12   |
| 0-55-123456-9 | Main Street     | 3     | 25   |
| 0-11-345678-9 | Moby Dick       | 3     | 49   |
| 0-12-333433-3 | On Liberty      | 1     | 25   |
| 0-11-354678-9 | Paris           | 3     | 29   |
| 0-123-45678-0 | Ulysses         | 2     | 34   |
| 1-22-233700-0 | Visual Basic    | 1     | 25   |

Table  
newLivres

- AS peut être utilisé dans la première instruction SELECT pour modifier les noms de colonnes retournées.
- Dans un UNION, on ne peut appliquer un ORDER BY pour chaque requête. Le ORDER BY doit être appliqué aux deux requêtes en même temps.
- La clause ORDER BY est utilisée à la fin de la dernière requête pour ordonner les données retournées des deux requêtes. Elle utilise toutefois les noms de colonnes de la première requête.
- GROUP BY et HAVING peuvent être utilisés dans chaque partie.
- UNION ne fait pas partie de SQL-92 mais est aussi présent dans Oracle et d'autres SGBD.

? 65-Q : Parle-moi de INTERSECT et EXCEPT (MINUS) ?

R :

- 1- **INTERSECT** représente l'intersection, elle fait ressortir le point commun entre les requêtes :

## Intersect

### ● Syntaxe

- SELECT ...  
**INTERSECT**  
SELECT ...

- Exemple:

```
SELECT t1.col1, t1.col2 FROM t1
INTERSECT
SELECT t2.col1, t2.col2 FROM t2
```

- 2- **EXCEPT** permet de ressortir la différence entre 2 requêtes :

# EXCEPT (MINUS)

- Pour exprimer la difference en SQL standard:

- `SELECT ...`  
**EXCEPT**  
`SELECT ...`

- Exemple:

```
SELECT t1.col1, t1.col2 FROM t1
```

```
EXCEPT
```

```
SELECT t2.col1, t2.col2 FROM t2
```

En oracle, on utilise le mot **MINUS**

- ? 66-Q : Donne un exemple d'utilisation de l'instruction UPDATE

R : *UPDATE Livres SET Livres.Prix = Livres.Prix + 1.00 WHERE Livres.Prix < 25.00*

Cette requête augmente de 1\$ le prix des livres dont le prix est inférieur à 25\$.

- ? 67-Q : Donne un exemple d'utilisation de l'instruction DELETE

R : *DELETE FROM LIVRES WHERE PubID="3"*

- ? 68-Q : Donne des exemples d'utilisation de l'instruction INSERT INTO

R :

1- *INSERT INTO LIVRES VALUES("1-000-00000-0","SQL",1,25.00);*

2- *INSERT INTO LIVRES(ISBN,Titre) VALUES("1-111-11111-1","Partie pêche");*

3- Pour insérer plusieurs champs à la fois :

- Supposons que NewLivres est une table de 3 champs: ISBN, PubID, et Prix.
- L'instruction suivante insère des lignes de LIVRES dans NewLivres. Elle insère uniquement les livres dont Prix > 20.00\$.

```
INSERT INTO NewLivres
SELECT ISBN, PubID, Prix
FROM Livres
WHERE Prix > 20;
```

**? 69-Q : Parle moi de l'instruction SELECT ... INTO**

**R :**

#### • L'instruction SELECT ... INTO

- L'instruction SELECT... INTO crée une nouvelle table et insère des données d'autres tables. La syntaxe est:  

```
SELECT NomChamp,...
INTO NomNouvelleTable
FROM Source
WHERE Condition
ORDER BY ConditionOrdre
```
- NomChamp est le nom du champ à copier dans la nouvelle table.
- Source est le nom de la table d'où proviennent les données. On peut mettre aussi une requête ou une jointure.

- Par exemple, l'instruction suivante crée une nouvelle table appelée LivresChers ET INCLUT LES LIVRES DE LA TABLE livres qui valent plus de 45.00\$:

```
SELECT Titre, ISBN
INTO LivresChers
FROM Livres
WHERE Prix > 45
ORDER BY Titre;
```

**? 70-Q : Donne un exemple de la clause NOT IN, IN, NOT EXISTS, EXISTS**

R :

- Syntaxe 2

Expression [NOT] IN (RequeteSQL)

- Cette syntaxe est utilisée pour consulter une valeur de colonne dans la table résultat d'une autre requête.
- Par exemple, la déclaration suivante retourne tous les titres de la table LIVRES qui n'apparaissent pas dans la table Livres2:

```
SELECT Titre FROM Livres
```

```
WHERE Titre NOT IN (SELECT Titre FROM Livres2);
```

- Syntaxe 3

[NOT] EXISTS (RequeteSQL)

- Cette syntaxe est utilisée pour vérifier si un item existe (si retourné) dans la requête secondaire.

- Par exemple, l'instruction suivante sélectionne tous les éditeurs qui n'ont pas de livres dans la table LIVRES:

```
SELECT PubNom FROM Editeurs WHERE NOT EXISTS
(SELECT * FROM Livres WHERE Livres.PubID =
Editeurs.PubID);
```

- Notes:

- Avec les syntaxes 1 ou 3, la requête secondaire doit retourner une seule colonne sinon il y a erreur.
- L'instruction SELECT d'une requête secondaire obéit au même format et aux mêmes règles que toute instruction SELECT. Toutefois, elle doit être placée entre parenthèse.

**? 71-Q : Les clauses INTERSECT et EXCEPT ne sont pas généralement supporté par les SGBD, comment faire alors ?**

R : Solution :

1- Pour INTERSECT on utilise **IN** :



- Avec la syntaxe 2, on peut définir l'intersection et la différence (vues en algèbre)
- Intersection  $\text{Table1} \cap \text{table2}$  :
  - **SELECT** attribut1, attribut2, ... **FROM** table1  
**WHERE** attribut1 **IN** (**SELECT** attribut1 **FROM** table2) ;
  - **SELECT** n°enseignant, NomEnseignant **FROM** E1  
**WHERE** n°enseignant **IN** (**SELECT** n°enseignant **FROM** E2) ;

2- Pour EXCEPT on utilise **NOT IN** :

- Difference  $\text{Table1} - \text{Table2}$ :
  - **SELECT** attribut1, attribut2, ... **FROM** table1  
**WHERE** attribut1 **NOT IN** (**SELECT** attribut1 **FROM** table2) ;
  - **SELECT** n°enseignant, NomEnseignant **FROM** E1  
**WHERE** n°enseignant **NOT IN** (**SELECT** n°enseignant **FROM** E2) ;

? 72-Q : Parle moi des Vues

R :



- Les vues permettent d'assurer l'objectif d'**indépendance logique**. Grâce à elles, chaque utilisateur pourra avoir sa vision propre des données.
- Les utilisateurs pourront consulter la base, ou modifier la base (avec certaines restrictions) à travers la vue, c'est-à-dire manipuler la table résultat du SELECT comme si c'était une table réelle.

### ● Syntaxe:

- CREATE VIEW nom\_vue [(nom\_col1,...)] AS SELECT ... [WITH CHECK OPTION] ;
- Une fois créée, une vue s'utilise comme une table. Il n'y a pas de duplication des informations mais stockage de la définition de la vue.

### ● Exemple:

```
CREATE VIEW emp10 AS SELECT * FROM emp WHERE n_dept = 10 ;
```

- si la vue emp10 a été créée avec CHECK OPTION on ne pourra à travers cette vue ni modifier, ni insérer des employés ne faisant pas partie du département 10

- Il est possible d'effectuer des INSERT et des UPDATE à travers des vues, sous deux conditions :
  - le SELECT définissant la vue ne doit pas comporter de jointure,
  - les colonnes résultats du SELECT doivent être des colonnes réelles et non pas des expressions.
- **Exemple** : Modification des salaires du département 10 à travers la vue emp10.  
UPDATE emp10 SET sal = sal \* 1.1;